Пермский институт (филиал) федерального государственного бюджетного
образовательного учреждения высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Отделение информационных технологий и программирования

Практическая работа №1

Тема работы: “Модульное тестирование в Visual Studio”

Работу выполнил:
Щеглов Даниил Никитич
Группа: ИПо-31
Преподаватель: Берестов Дмитрий Борисович

Пермь 2026

СОДЕРЖАНИЕ

1. Введение: задачи практики 3

2. Что я разрабатывал 3

3. Организация проекта 3

4. Процесс тестирования: от первого запуска до рефакторинга 4

4.1 Базовый тест на корректное списание 4

4.2 Проверка обработки некорректных входных данных 5

4.3 Границы значений и параметризованные тесты 5

4.4 Рефакторинг: сообщения об ошибках в константах 6

4.5 Случай, когда ошибка оказалась в тесте 7

5. Итоги выполнения 7

6. Чему я научился 8

7. Ссылки 9

1. Введение: задачи практики

В рамках практической работы я освоил создание модульных тестов для управляемого кода на платформе .NET с использованием фреймворка MSTest. Задачи, которые я ставил перед собой: научиться создавать тестовый проект и подключать к нему тестируемую библиотеку; писать тесты по схеме «подготовка — действие — проверка»; проверять ожидаемые исключения; применять параметризованные тесты для набора однотипных значений; читать отчёт о прогоне и отличать ошибку в коде от ошибки в самом тесте.

2. Что я разрабатывал

Мне предстояло разработать и протестировать класс BankAccount, моделирующий банковский счёт. Класс хранит имя владельца, баланс, признак закрытия и журнал операций. Открытый интерфейс включает метод Credit() для пополнения счёта, метод Debit() для списания, метод CanDebit() для проверки возможности списания без изменения состояния, метод TransferTo() для перевода между счетами и метод Close() для закрытия счёта.

Баланс доступен только для чтения, поэтому изменить его можно исключительно через операции, каждая из которых проверяет свои предусловия: состояние счёта, знак суммы и достаточность средств. Для сумм я использовал тип decimal. Тип double хранит числа в двоичном виде и не может точно представить 0,1, из-за чего при сложении накапливается погрешность; decimal хранит мантиссу в десятичной системе, и сумма 0,1 + 0,2 даёт ровно 0,3.

3. Организация проекта

Я создал решение ScheglovBank, в которое включил три проекта. Первый проект, Scheglov.Bank, содержит библиотеку классов с тестируемой логикой. Второй, Scheglov.Bank.Tests, представляет собой проект модульных тестов на

базе MSTest. Третий, Scheglov.Bank.Demo, — консольное приложение, которое показывает работу со счётом и печатает выписку.

Все проекты ориентированы на .NET 8. В тестовом проекте подключены пакеты Microsoft.NET.Test.Sdk, MSTest.TestAdapter и MSTest.TestFramework версии 3.6.4, а также указана ссылка на тестируемую библиотеку. Тесты запускаются из обозревателя тестов Visual Studio или командой dotnet test; второй способ удобен тем, что прогон можно включить в автоматическую сборку.

4. Процесс тестирования: от первого запуска до рефакторинга

Работа шла итерациями: я писал очередную группу тестов, запускал прогон и по его результатам либо исправлял код, либо уточнял ожидания в тесте.

4.1 Базовый тест на корректное списание

Первым я написал тест, проверяющий штатную ситуацию: списание допустимой суммы должно уменьшать баланс. При начальном балансе 20,00 и сумме списания 6,35 ожидаемый остаток равен 13,65. Тест прошёл сразу, так как логика списания была реализована корректно.

```
[TestMethod]
public void Debit_WithValidAmount_ReducesBalance()
{
    var account = new BankAccount("Щеглов Даниил", 20.00m);

    account.Debit(6.35m);

    Assert.AreEqual(13.65m, account.Balance);
}
```

Отдельным тестом я проверил границу: списание суммы, равной всему балансу, должно оставлять ноль, а не отрицательное значение. Именно на таких сравнениях чаще всего и возникают ошибки.

4.2 Проверка обработки некорректных входных данных

Следующим шагом я проверил поведение метода при недопустимых суммах. Отрицательная сумма и сумма, превышающая остаток, должны приводить к исключению ArgumentOutOfRangeException с понятным сообщением. Для проверки я использовал метод Assert.ThrowsException: в отличие от конструкции try/catch он самостоятельно фиксирует падение теста, если исключение не было выброшено, и возвращает перехваченный объект, что позволяет дополнительно проверить текст сообщения.

```
[TestMethod]
public          void          Debit_WhenAmountExceedsBalance_ThrowsExpectedMessage()
{
    var     account     =     new     BankAccount("Щеглов     Даниил",     20.00m);

    var    error    =    Assert.ThrowsException<ArgumentOutOfRangeException>(()    =>
account.Debit(20.01m));

    StringAssert.Contains(error.Message,
BankAccount.DebitAmountExceedsBalanceMessage);
    Assert.AreEqual(20.00m,                                          account.Balance);
}
```

Обратите внимание на последнее утверждение: тест проверяет не только тип исключения, но и то, что баланс остался прежним. Без этой проверки тест прошёл бы и в случае, когда сумма списывается до проверки условия, а исключение формируется уже после изменения состояния.

4.3 Границы значений и параметризованные тесты

Для суммы списания я выделил классы эквивалентности и граничные значения. Проверка границ важна потому, что ошибки в операциях сравнения проявляются именно там: например, при сумме, в точности равной балансу.

Область значений	Класс	Ожидаемое поведение
amount < 0	недопустимый	ArgumentOutOfRangeException
amount = 0	допустимый (граница)	баланс не меняется
0 < amount < balance	допустимый	баланс уменьшается на сумму
amount = balance	допустимый (граница)	баланс становится равным нулю
amount > balance	недопустимый	ArgumentOutOfRangeException

Таблица 1 — Классы эквивалентности для суммы списания

Однотипные проверки я вынес в параметризованный тест: набор пар «сумма — ожидаемый результат» задаётся атрибутами DataRow, а тело теста остаётся общим. Это уменьшает дублирование и упрощает добавление новых граничных значений.

```
[DataTestMethod]
[DataRow(5,                                                                    true)]
[DataRow(20,                                                                   true)]
[DataRow(20.01,                                                               false)]
[DataRow(-1,                                                                  false)]
public void CanDebit_ReturnsAvailabilityWithoutChangingBalance(double amount, bool
expected)
{
    var     account     =     new     BankAccount("Щеглов     Даниил",     20.00m);

    var            actual           =            account.CanDebit((decimal)amount);

    Assert.AreEqual(expected,                                                  actual);
    Assert.AreEqual(20.00m,                                          account.Balance);
}
```

4.4 Рефакторинг: сообщения об ошибках в константах

Чтобы тесты не зависели от точных формулировок, я вынес тексты сообщений в публичные константы класса. Теперь тест ссылается на константу, а не на строковый литерал, поэтому изменение текста не потребует правки тестов.

```
public  const  string  DebitAmountExceedsBalanceMessage  =  "Debit  amount  exceeds
balance";
public const string DebitAmountLessThanZeroMessage = "Debit amount is less than
zero";
public const string CreditAmountLessThanZeroMessage = "Credit amount is less than
zero";

//                                                                              ...

throw        new        ArgumentOutOfRangeException(nameof(amount),        amount,
DebitAmountLessThanZeroMessage);
```

Дополнительно я передаю в конструктор исключения имя аргумента и его значение. Благодаря этому сообщение содержит не только причину отказа, но и конкретное значение, которое к нему привело, — это заметно упрощает отладку.

4.5 Случай, когда ошибка оказалась в тесте

Отдельного внимания заслуживает эпизод, когда упавший тест указал на ошибку не в коде, а в моих ожиданиях. Я проверял, что при неудачном переводе состояние обоих счетов остаётся неизменным, и дополнительно сравнивал количество записей в журналах.

Первый прогон дал 23 успешных теста из 24. Упало утверждение о количестве записей в журнале получателя: я ожидал ноль, а тест получил единицу. Причина оказалась в том, что счёт получателя открывался с ненулевым балансом, и при открытии в журнал добавляется запись с видом Opening. Реализация была права, ошибка была в ожидании.

```
// Было (неверное ожидание): получатель открыт с балансом 10, запись Opening уже
есть
Assert.AreEqual(0,                                    receiver.Transactions.Count);

// Стало (верно): запись об открытии счёта существует независимо от перевода
Assert.AreEqual(1, receiver.Transactions.Count);
```

Этот случай хорошо показывает, почему падение теста — ещё не приговор программе. Сначала нужно понять, какая из сторон ошибается: код или проверка. Слепо подгонять код под тест так же неправильно, как подгонять тест под код.

5. Итоги выполнения

В результате набор содержит 24 теста, и все они проходят успешно; прогон занимает около 0,05 секунды. Покрыты создание счёта и проверка аргументов конструктора, пополнение и списание, включая граничные суммы, проверка доступности средств, точность десятичных вычислений, журнал операций с остатком после каждой операции, перевод между счетами и его ошибочные варианты, а также работа с закрытым счётом.

```
dotnet            test            ScheglovBank.sln            -c            Release

Пройден! : не пройдено 0, пройдено 24, пропущено 0, всего 24
```

Консольная демонстрация показывает типичный сценарий работы со счётом: открытие, пополнение, списание, печать выписки и обработку отклонённой операции.

```
Владелец:                                Щеглов                                Даниил
Текущий                                 баланс:                                 12,75

Выписка                                   по                                  счёту:
  09:14:02       Opening       +           15,50       баланс:                 15,50
  09:14:02      Credit          +             4,50      баланс:                 20,00
  09:14:02      Debit             -            7,25      баланс:                 12,75

Операция   отклонена:   Debit   amount   exceeds   balance   (Parameter   'amount')
Actual value was 100.
```

6. Чему я научился

В ходе работы я закрепил следующие навыки:

– создание и настройка тестового проекта MSTest в Visual Studio;

– использование атрибутов TestClass, TestMethod, DataTestMethod и TestInitialize;

– применение утверждений Assert.AreEqual, Assert.ThrowsException и StringAssert.Contains;

– выделение классов эквивалентности и граничных значений для входных данных;

– чтение отчёта о прогоне и определение причины падения теста;

– различение ошибки в коде и ошибки в ожиданиях теста;

– использование типа decimal для денежных сумм.

Модульное тестирование показало свою эффективность: набор тестов обнаруживает проблему сразу после изменения кода и защищает проект от регрессий. При дальнейшем развитии работы имеет смысл добавить тесты на многопоточный доступ к счёту и проверку сохранения счетов в базе данных.

7. Ссылки

1. Модульное тестирование в .NET с помощью MSTest. — URL: https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest

2. Обзор платформы модульного тестирования MSTest. — URL: https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-dotnet-test

3. Рекомендации по написанию модульных тестов. — URL: https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-best-practices

4. Справочник по C#. Тип decimal. — URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types